

FEEDBACK

Tema a avut un grad de dificultate mediu, insa componenta de scalabilitate a necesitat un efort considerabil comparativ cu asigurarea corectitudinii.

O provocare majora a fost interpretarea corecta a timpilor de executie pe masina locala. Am observat fluctuatii semnificative cauzate de mecanismele sistemului de operare. De exemplu, rularile succesive erau mult mai rapide initial, deoarece fisierele de intrare ramaneau probabil incarcate in memoria cache, eliminand astfel latenta citirii de pe disc.

Din acest motiv, consider ca accesul la un mediu controlat (VM/Docker) ar fi ajutat la o evaluare mai obiectiva a optimizarilor.

Timp de lucru: aproximativ 5 zile.

STRATEGIA DE PARALELIZARE

Programul este structurat ca un pipeline de date, unde un set fix de thread-uri (P) proceseaza datele in etape secventiale, sincronizate prin bariere.

Modelul de partitionare a muncii: Am folosit o distributie statica a sarcinilor, intervalul de munca a unui thread fiind calculat dupa id-ul sau, folosind formulele:

```
int start = ID * (double) N / P;  
int end = min((ID + 1) * (double) N / P, N);
```

Etapele de procesare:

1. parseArticles

- **Ce face:** Citeste fisierele JSON, extrage campuril de interes, salveaza rezultatul intr-o lista
- **Paralelizat:** Aici N este numarul total de fisiere. Fiecare thread parseaza independent fisierele din intervalul sau.
- **Sincronizare (Thread-Safe):** Variabila partajata allArticles este o Collections.synchronizedList. Aceasta permite thread-urilor sa adauge articole simultan.
- **Serializat (Single Thread):** Niciuna.

2. removeDuplicates

- **Ce face:** Elimina duplicatele si populeaza lista finala.
- **Paralelizat:** Aici N este numarul de articole brute. Thread-urile parcurg intervalul lor pentru a numara aparitiile si apoi pentru a filtra, aceste 2 etape fiind separate de o bariere.
- **Sincronizare (Thread-Safe):** Variabilele uuidCounts si titleCounts sunt de tip ConcurrentHashMap, permitand incrementarea atomica (merge) fara blocaje majore. Lista finalArticles este si ea sincronizata.
- **Serializat (Single Thread):** Niciuna.

3. organizeArticlesIntoCategories & organizeArticlesIntoLanguages

- **Ce face:** Distribuie UUID-urile in map-urile globale.
- **Paralelizat:** Aici N este numarul de articole unice. Fiecare thread proceseaza articolele din zona sa.
- **Sincronizare (Thread-Safe):** Variabilele CategoriesMap si LanguageMap sunt ConcurrentHashMap. Mai mult, valorile din aceste mape sunt liste de tip SynchronizedList, ceea ce permite ca doua thread-uri sa adauge UUID-uri in aceeasi categorie in acelasi timp, fara a corupe lista.
- **Serializat (Single Thread):** Niciuna.

4. writeCategoryFiles & writeLanguageFiles

- **Ce face:** Scrie fisierele text de iesire.
- **Paralelizat:** Aici N este numarul de chei (categorii/limbi). Thread-urile isi impart responsabilitatea numelor de fisiere. Deoarece seturile de fisiere sunt disjuncte, scrierea pe disk este complet paralela.
- **Sincronizare (Thread-Safe):** Nu este necesara, deoarece nu exista resurse partajate la scriere.
- **Serializat (Single Thread):** Niciuna.

5. writeGlobalSortedArticles

- **Ce face:** Sorteaza si scrie fisierul all_articles.txt.
- **Paralelizat:** Sortarea initiala a listelor locale. Aici N este numarul de articole unice.
- **Sincronizare (Thread-Safe):** Listele partiale sunt adaugate intr-o structura comuna sortedListsArticle (sincronizata).
- **Serializat (Single Thread):** Etapa de Merge (Interclasare) si scrierea efectiva in fisier. Logica de alegere a articolului maxim din listele sortate si scrierea lui este

executata exclusiv de Thread 0 (if (id == 0)), pentru a garanta ordinea cronologica stricta in fisierul final.

6. processKeywords

- **Ce face:** Analizeaza frecventa cuvintelor in articolele in engleza.
- **Paralelizat:** Filtrarea textului. Aici N este numarul de articole unice.
- **Sincronizare (Thread-Safe):** Variabila keywordCounts (ConcurrentHashMap) este actualizata concurrent folosind operatii atomice.
- **Serializat (Single Thread):** Niciuna.

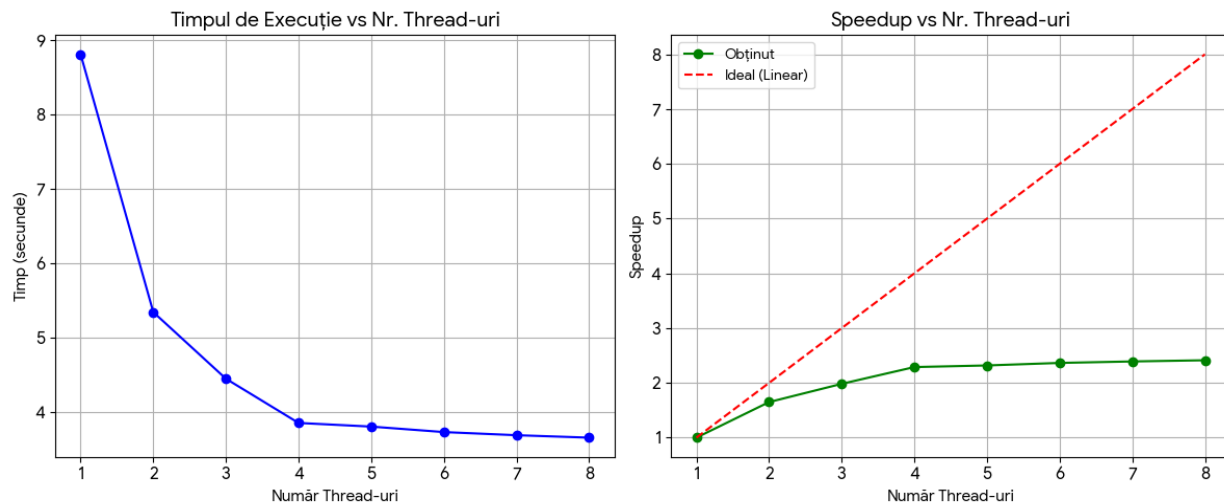
7. writeKeywordsFile (Sortarea și scrierea cuvintelor)

- **Ce face:** Sortează și scrie fișierul keywords_count.txt.
- **Paralelizat:** Sortarea parțială. Fiecare thread preia responsabilitatea pentru un subset de cuvinte, determinat prin formula $\text{abs}(\text{hash}(\text{cuvant}) \% P) == \text{ID}$.
- **Sincronizare:** Listele sortate local sunt adăugate în sortedListsKeyword (sincronizată).
- **Serializat:** Interclasarea listelor și scrierea finală de către Thread 0.

8. processReports & writeReports

- **Ce face:** Calculeaza statistici si scrie raportul final.
- **Paralelizat:** Calculul statisticilor in variabile locale. Aici N este numarul de articole unice.
- **Sincronizare (Thread-Safe):** Agregarea variabilelor locale in cele globale (authorCounts, languageCounts) se face prin operatii thread-safe la finalul etapei paralele.
- **Serializat (Single Thread):** Scrierea fisierului reports.txt. Doar Thread 0 executa aceasta secventa pentru a respecta formatul cerut.

ANALIZA DE PERFORMANTA SI SCALABILITATE



Setup de testare

Setul de date ales a fost cel de la test_5, fiind cel mai mare si mai sugestiv. Testele au fost efectuate pe urmatorul sistem:

- **CPU:** 12th Gen Intel(R) Core(TM) i5-12500H (12 Cores / 16 Threads)
- **RAM:** 16 GB
- **OS:** Ubuntu 22.04.5 LTS
- **Java:** OpenJDK 17.0.17

Metodologie

Pentru a asigura acuratetea masuratorilor si a elimina latentă cauzată de Disk I/O (citirea lentă de pe disc), am efectuat o rulare de 'warm-up' înainte de măsurătorile efective. Astfel, timpurile raportate reflectă performanța algoritmului de procesare paralelă (CPU-bound) având datele încărcate în memoria cache sistemului de operare. Fiecare test a fost repetat de 3 ori, iar în tabel este notată media aritmetică a timpilor.

Threads (p)	Timp Executie (s)	Speedup $S(p)=T(1)/T(p)$	Eficiența $E(p)=S(p)/p$
1	8.813	1.00	100%
2	5.342	1.65	82.5%
3	4.445	1.98	66.1%

4	3.850	2.29	57.2%
5	3.801	2.32	46.4%
6	3.727	2.37	39.4%
7	3.685	2.39	34.2%
8	3.652	2.41	30.2%

Analiza si Concluzii

Se observa o crestere a performantei (Speedup) semnificativa la trecerea de la 1 la 4 thread-uri, unde timpul de executie scade de la 8.8s la 3.85s, obtinand un Speedup de **2.29x**.

Incepand cu 5 thread-uri si continuand pana la 8, curba de accelerare intra intr-o zona de saturatie (Speedup maxim ~2.41x). Diferenta de timp intre 4 thread-uri (3.85s) si 8 thread-uri (3.65s) este de doar 0.2 secunde, in ciuda dublarii numarului de fire de executie. Eficienta scade drastic, de la 57% la 30%.

Cauzele limitarii:

1. **Arhitectura Hibrida a Procesorului:** Procesorul i5-12500H are o arhitectura hibrida cu **4 nuclee de Performanta (P-cores)** si 8 nuclee de Eficienta (E-cores). Scalabilitatea este foarte buna pana la 4 thread-uri deoarece acestea sunt alocate pe nucleele puternice (P-cores). Odata depasit acest numar, sistemul de operare este fortat sa aloce thread-urile suplimentare pe nucleele E (mai lente) ceea ce duce la castiguri marginale de performanta.
2. **Limita de viteza secventiala (Legea lui Amdahl):** Programul are etape care nu pot fi paralelizate, cum ar fi scrierea finala a rezultatelor in fisierele text (realizata doar de Thread-ul 0). Pe masura ce partea de procesare devine extrem de rapida (datorita celor 8 thread-uri), timpul fix necesar scrierii pe disc devine procentual mult mai mare, impiedicand scaderea timpului total sub un anumit prag.
3. **Aglomeratia pe resurse comune (Overhead):** Deoarece toate thread-urile scriu simultan in aceleasi structuri de date (ConcurrentHashMap si liste sincronizate), apar mici blocaje temporare. Cand un thread vrea sa adauge un articol in lista,

celelalte trebuie sa astepte scurt timp. Cu cat avem mai multe thread-uri (8), cu atat aceste asteptari se aduna si consuma resurse, anuland castigul de viteza.

Concluzie: Scalabilitatea optima pe sistemul de test este atinsa la **4 thread-uri**, coincizand cu numarul de nuclee fizice de performanta ale procesorului. Utilizarea unui numar mai mare de thread-uri este ineficienta, consumand resurse pentru un castig de performanta neglijabil (< 5%).